Trabalho de Conclusão de Curso de Engenharia da Computação

PROTÓTIPO PARA DETECÇÃO, LOCALIZAÇÃO E SELEÇÃO AUTOMÁTICA DE AÇÃO POR SENSORIAMENTO:
MODELO DE DADOS

Prototype for automatic action detection, localization and selection through sensing:
Data model

Gabriel Vasconcellos de Morais[1]
Matheus Emanoel Gomes Souza[2]
Anker Douglas Loss[3]

RESUMO

A construção de protótipos de sensoriamento com hardware acessível é um recurso didático relevante em Engenharia da Computação, por permitir explorar de forma prática conceitos de aquisição de dados. Integrar sensoriamento de baixo custo, processamento em tempo real e seleção automática de resposta em um único protótipo exige lidar com comunicação entre hardware e software e tratamento de leituras ruidosas. Este trabalho tem como objetivo demonstrar a viabilidade de um sistema de detecção, localização e seleção automática de ação construído inteiramente com componentes acessíveis, em vez de hardware industrial. O sistema combina dois sensores ultrassónicos (módulo de baixo custo que mede a distância pelo eco do ultrassom) com quatro câmeras de 720p, processa quadros de vídeo (*frames*) e classifica o alvo por sua cor, formato e tamanho. O protótipo resultante, denominado RadarTorres, converte leituras de ângulo e distância em posição cartesiana, exibindo-as em tempo real em um radar circular, seleciona automaticamente, entre torres candidatas posicionadas ao redor da base, a mais adequada para responder a cada alvo, e registra em auditoria toda ação executada. O processamento e a tomada de decisão são concentrados em uma aplicação desktop, com arquitetura pensada para permitir evolução futura sem exigir hardware adicional. Os requisitos do sistema foram levantados por engenharia reversa diretamente do código-fonte já implementado. Discutem-se, por fim, as limitações inerentes à abordagem de baixo custo adotada.

Palavras-chave: Sensoriamento de baixo custo; Detecção e rastreamento de alvos; Seleção automática de ação; Arduino; Protótipo.

ABSTRACT

[1]Graduando(a) em Engenharia da Computação pela Faculdade do Centro Leste – UCL. E-*mail*: gabrielmorais@ucl.br
[2]Graduando(a) em Engenharia da Computação pela Faculdade do Centro Leste – UCL. E-*mail*: matheusemanoel@ucl.br
[3]Orientador(a). Mestre em xxx pela xxx. E-*mail*: anker@ucl.br

This work describes the RadarTorres prototype, a system for automatic action detection, localization and selection through sensing, developed as a Final Course Project in Computer Engineering. The prototype integrates sensors to identify objects in a monitored environment, register their position and allow manual or automatic activation of response devices. The system combines two ultrasonic sensors (a low-cost module that measures distance by ultrasound echo) with four 720p cameras, processes video frames, and classifies the target by color, shape, and size. This work details the application's data model: persistence is currently implemented through CSV files organized per operating-system user, covering six tables – users, detected objects, executed actions, mode changes, user preferences and help tickets –, each accessed through a dedicated repository interface. This choice, documented in the project's technical records, keeps the system simple at this stage while explicitly signaling the intent to migrate to a relational database in the future. The current table structure, its relationships (by login, without real foreign keys) and the proposed model for the SQL migration are described, in which textual references would be replaced by proper foreign keys, without requiring changes to the application's ViewModel or Service layers, since they depend only on the repository interfaces. Results indicate that the interface-based architecture enables this migration incrementally and with low risk.

Keywords: Sensing; Data model; Persistence; SQL migration; Prototype.

1 Introdução

Sistemas automáticos de detecção e resposta a alvos exigem, além de identificar um estímulo, rastrear com precisão sua posição e aplicar regras de decisão que evitem respostas indevidas – tarefas comumente resolvidas com sensores de radar ou LIDAR, de custo elevado. Como alternativa a esse tipo de equipamento, este trabalho adota técnicas de visão computacional, aliadas a hardware acessível, como parte central da arquitetura do protótipo. Protótipos didáticos construídos dessa forma desempenham papel importante nesse processo, por permitirem que esses conceitos sejam explorados de forma prática e replicável em laboratório, reproduzindo os princípios de sensoriamento, rastreamento e decisão presentes em sistemas mais sofisticados.

2 Desenvolvimento

2.1 Revisão da literatura e fundamentação teórica

Sistemas de detecção e rastreamento de alvos costumam recorrer a sensores como radar e LIDAR, de custo elevado. Como alternativa, sensores de baixo custo podem ser combinados de modo que as limitações de um sejam compensadas pelas qualidades do outro. Sensores ultrassônicos, como o HC-SR04, medem distância com baixo custo e boa confiabilidade em curto alcance, mas não distinguem o tipo do obstáculo; câmeras, por sua vez, fornecem informação semântica, porém dependem das condições de iluminação e exigem maior poder de processamento (**??**). A fusão de sensores (*sensor fusion*) combina as duas modalidades para aproveitar essa complementaridade.

??) descrevem uma arquitetura em que um Arduino Mega trata o sensoriamento ultrassônico de baixa latência, enquanto um Raspberry Pi processa as imagens da câmera com uma rede YOLOv4-tiny, com comunicação serial entre os dois. Essa divisão de papéis é análoga à do RadarTorres, em que o Arduino realiza a aquisição e a aplicação desktop concentra o processamento. A literatura também mostra que sensores ultrassônicos de baixo custo podem complementar câmeras e LiDAR em sistemas de rastreamento, com a precisão melhorada por predição baseada em filtro de Kalman e fusão com a distância medida pelos ultrassônicos (**??**). O mesmo trabalho observa que as leituras ultrassônicas se tornam ruidosas em distâncias maiores, o que motiva o uso de filtragem.

Definidos o alvo e sua posição, resta decidir qual recurso deve responder. Na literatura de defesa, esse problema é tratado como avaliação de ameaças e atribuição de armas (*Threat Evaluation and Weapon Assignment*, TEWA), no qual as informações dos sensores alimentam as etapas de detecção, rastreamento e identificação, seguidas da avaliação da ameaça e da atribuição dos recursos (**????**). As abordagens são classificadas em ameaça por ameaça, em que as armas são atribuídas sequencialmente, e multiameaça, em que o conjunto de ameaças é considerado simultaneamente (**??**).

O RadarTorres adota uma abordagem simples, por proximidade, aplicada alvo a alvo, com propósito exclusivamente demonstrativo (laser de baixa potência, LED ou simulação, sem armamento real); a abordagem multiameaça fica como trabalho futuro.

O refinamento dessa escolha por proximidade – e o instante de disparo – é feito por uma pequena rede neural *feedforward* (6 entradas, 1 camada oculta de 8 neurônios, 2 saídas, ativação tanh), cujos pesos não são ajustados por retropropagação, mas evoluídos por um algoritmo genético próprio (seleção por torneio, *crossover* uniforme, mutação gaussiana, elitismo), auditável e sem dependência de biblioteca de aprendizado de máquina. **??**) mostram que algoritmos genéticos são uma alternativa competitiva ao treino por gradiente em tarefas de aprendizado por reforço, inclusive em redes bem maiores que a aqui empregada, o que sustenta a escolha de ampliar o desempenho do controlador pelo ajuste dos hiperparâmetros da busca evolutiva (tamanho de população, paralelização da avaliação de fitness, enriquecimento do sinal de recompensa) em vez de aumentar o número de camadas ou neurônios sem evidência de que a capacidade atual da rede seja o fator limitante.

O impacto do treinamento virtual sobre a execução física do controlador é atenuado por uma decisão de projeto central: a mesma classe que decide no episódio simulado (`ControladorHibrido`) é a que decide em produção, consultada por `TowerSelectionService` a partir das leituras do Arduino real – não há conversão para um formato intermediário (ONNX, TensorFlow Lite) nem reimplementação em outra linguagem ou plataforma, fontes comuns de divergência entre o comportamento treinado e o executado em sistemas de aprendizado por reforço aplicados a robótica. O genoma evoluído é um vetor plano de pesos e vieses, embutido diretamente na aplicação (`EmbutirGenomaTreinado.ps1`, `IControladorTreinadoProvider`); o que muda do ambiente de treino para a operação real é apenas a origem dos dados de entrada (leitura simulada ou leitura do Arduino), nunca a função de decisão em si. Os parâmetros que definem o episódio simulado – alcance de engajamento e certeza mínima para autorizar disparo – também deixaram de ser constantes fixas no código e passaram a ser lidos das mesmas configurações usadas pelo sistema real (`ConfiguracoesAplicativo.AlcanceMiraMetros`, `LimiarConfiancaDisparoPercentual`), de modo que recalibrar o alcance da câmera ou o limiar de certeza no painel de Configurações recalibra também o mundo de treino, reduzindo a divergência entre o que foi aprendido e o que será executado. Por fim, a rede neural nunca decide isoladamente: ela apenas pondera a escolha dentro do que a camada heurística determinística já considera válida (nunca uma torre ocupada, nunca fora de alcance, nunca dentro de uma zona morta), o que limita o pior caso de uma generalização imperfeita do treino virtual para a física – no limite, o sistema regride ao comportamento puramente heurístico, nunca a uma decisão arbitrária.

2.2 Procedimentos metodológicos

O percurso metodológico deste trabalho não seguiu o fluxo tradicional de especificação prévia seguida de implementação: o protótipo RadarTorres já existia, em desenvolvimento incremental, antes do levantamento formal de requisitos. Optou-se, portanto, por uma abordagem de engenharia reversa documental, na qual os requisitos funcionais e não funcionais foram extraídos diretamente do código-fonte já implementado (`src/RadarTorres.App/`), do firmware de simulação (`Arduino/ArduinoSimulation.ino`) e da documentação técnica já produzida ao longo do projeto, em vez de definidos a priori em um documento de especificação anterior à codificação.

Esse levantamento resultou em 25 requisitos funcionais (RF01–RF25) e 20 requisitos não

funcionais (RNF01–RNF20), classificados segundo dois critérios objetivos: a **prioridade**, atribuída conforme a criticidade de cada requisito para a operação central do sistema (comunicação serial, rastreamento, seleção de torre, modos de operação e segurança recebem prioridade Alta; suporte administrativo e ferramental, Média; conveniências de interface, Baixa); e o **status**, que reflete o estado real de implementação constatado no código no momento do levantamento – Implementado, Parcial ou Planejado –, e não a intenção original de projeto. Cada requisito foi, além disso, rastreado até a classe ou método do código-fonte que o realiza, registro mantido em uma matriz de rastreabilidade dedicada, de modo que qualquer afirmação sobre o grau de implementação do protótipo possa ser verificada diretamente no código. Divergências encontradas entre o comportamento observado no código e a nomenclatura ou fluxo originalmente esperado também foram registradas explicitamente, em vez de omitidas ou corrigidas silenciosamente na documentação.

A validação funcional do protótipo, por sua vez, não foi realizada sobre o hardware físico, ainda não montado: o sistema conta com um modo de simulação embutido na própria aplicação, que gera leituras sintéticas de ângulo e distância no mesmo formato produzido pelo Arduino, permitindo exercitar de ponta a ponta o rastreamento de alvos, a seleção automática de torre e as regras de zona morta sem depender de uma porta serial conectada. Esse modo de simulação foi o meio primário usado para observar o comportamento do sistema durante este trabalho, o que impõe um limite claro ao escopo dos resultados relatados: eles atestam a corretude da lógica de decisão implementada em software, mas não substituem uma validação de campo com sensores e torres físicas, deixada como trabalho futuro.

A mesma ressalva se aplica, com reforço adicional, ao controlador treinado por neuroevolução: o mundo simulado em que os genomas são avaliados (`AmbienteTreinamento`) é uma aproximação idealizada do ambiente físico – os alvos nascem na borda de um círculo e avançam de forma quase radial, sem o ruído típico de leituras ultrassônicas reais; a certeza de identificação do alvo pela câmera é modelada como uma função sintética que cresce com o tempo de observação, e a probabilidade de errar o disparo é uma função assumida, crescente com a velocidade do alvo – nenhuma delas calibrada com dados de campo, já que o hardware ainda não foi montado. Essa distância entre simulação e realidade física não é apenas teórica: durante este trabalho, identificou-se e corrigiu-se uma falha em que a certeza de identificação do alvo era sorteada uma única vez, no nascimento, e nunca era atualizada – cerca de 70% dos alvos simulados nasciam permanentemente abaixo do limiar mínimo de disparo e jamais podiam ser abatidos, independentemente da qualidade do genoma, mascarando qualquer evolução real do controlador sob a aparência de estagnação. O episódio é citado aqui como evidência concreta de que uma premissa equivocada na simulação pode invalidar silenciosamente o processo de treinamento sem que isso seja percebido pelas métricas agregadas, reforçando a necessidade de validação cruzada entre o comportamento do genoma treinado e o comportamento observado em campo, com hardware físico, antes de se confiar no controlador em um cenário de operação real – validação que permanece como trabalho futuro, nos mesmos termos já apontados para o restante do protótipo.

2.3 Projeto e implementação

2.3.1 Módulo de detecção de alvos

O módulo de detecção utiliza dois sensores ultrassônicos HC-SR04 instalados de costas um para o outro no topo de um mastro, na torre central. O mastro é movido por um motor de passo 28BYJ-48 que oscila 180°, de modo que, em conjunto, os dois sensores cobrem uma varredura de 360°. Cada sensor emite pulsos de 40 kHz e calcula a distância pelo atraso do eco, segundo a relação $d = \frac{v \cdot \Delta t}{2}$, em que v é a velocidade do som no ar (cerca de 343 m/s à temperatura ambiente). A cada leitura, o Arduino envia à aplicação desktop, pela porta serial, o ângulo e a distância em protocolo de texto simples, que a aplicação converte em posição cartesiana e exibe no radar circular de quatro quadrantes.

O protocolo é textual em ASCII, transportado pela porta serial USB a 9600 bauds por padrão, com cada mensagem terminada por `\n` e composta por um tipo em maiúsculas seguido de pares chave-valor separados por ponto e vírgula. Uma leitura de alvo chega na forma `TARGET;ID=1;ANGLE=45;DIST=2.80`, em que `ANGLE` é dado em graus no intervalo $[0, 360)$ e `DIST` em metros. O ângulo segue a convenção de bússola – 0° apontando para o Norte, sentido horário –, escolhida por ser a leitura mais intuitiva para quem opera um radar, e a conversão para coordenadas cartesianas de mundo, isolada em `CoordinateConverter.PolarToCartesian`, é dada por $x = d \cdot \sin\theta$ e $y = d \cdot \cos\theta$, com θ em radianos. A partir do par (x, y), o quadrante do alvo é determinado pelos sinais das duas coordenadas (`QuadrantHelper`) e a distância até cada torre candidata pela distância euclidiana clássica (`DistanceCalculator.Euclidean`). Leituras com campos ausentes, não numéricos, com ângulo fora do intervalo válido ou com distância negativa são convertidas em mensagens de erro registradas no console de eventos, nunca em exceções – falhas de sensor ou de cabo produzem degradação graciosa, e não interrupção da aplicação.

2.3.2 Projeto elétrico e estrutural

O projeto elétrico é baseado em um Arduino Uno, plataforma disponível no simulador Tinkercad usado para simular o circuito. Os sinais dos motores de passo são gerados por registradores de deslocamento 74HC595 em cascata, e os motores 28BYJ-48 são acionados por drivers ULN2003, alimentados por uma fonte de 5 V/15 A. Cada uma das quatro torres armadas possui dois motores 28BYJ-48 (movimentos de pan e tilt), um laser de baixa potência e um LED de cor própria; o acionamento é exclusivamente demonstrativo. Na torre central, o mastro dos sensores não é acoplado diretamente ao eixo do motor: ele assenta em um disco que gira sobre um rolamento axial 51108, acionado pelo motor por um cubo duplo-D. A passagem dos fios fica próxima ao eixo central, com folga de cabo em vez de anel coletor (*slip ring*), já que o giro é limitado a 180°. A torre central é cerca de 5 cm mais alta que as torres armadas (cerca de 137,5 mm de altura). As peças estruturais são impressas em 3D.

2.3.3 Módulo de visão

O módulo de visão utiliza quatro câmeras de 720p conectadas por USB ao computador, associadas aos quadrantes do radar, e processa os quadros de vídeo (*frames*) para classificar o alvo por sua cor, formato e tamanho. Cada quadro capturado é entregue já decodificado à detecção, que executa localmente um modelo tiny-YOLOv3 em formato ONNX para propor as regiões candidatas da imagem. Cada candidata é então comparada ao perfil do alvo previamente cadastrado – definido por cor, formato e tamanho, e opcionalmente por fotos de referência recortadas no objeto de interesse. Havendo foto de referência, a comparação combina um histograma de matiz e saturação com uma grade espacial de cor média, usados como assinatura do objeto; na ausência de foto, o critério recai sobre a fração de pixels com a cor cadastrada e sobre a classificação de formato. Quando o detector não propõe nenhuma caixa com a cor do perfil, as regiões candidatas passam a ser propostas por segmentação de cor, como último recurso na ordenação de candidatos.

O envio das informações resultantes ao Arduino, para guiar o apontamento da torre (movimentos de pan e tilt), foi projetado mas ainda não está implementado: o protocolo serial atual não possui comando de apontamento, e a fusão entre a posição estimada pela visão e a distância medida pelos sensores ultrassônicos permanece como etapa planejada. Como o hardware ainda não foi montado, a validação funcional descrita neste trabalho foi feita por simulação, e nenhum resultado de campo é relatado.

2.3.4 Fluxo operacional

Em operação, o sistema segue estas etapas: (1) o Arduino varre o ambiente com os dois sensores ultrassônicos e envia ângulo e distância à aplicação; (2) a aplicação converte cada leitura em posição cartesiana, determina o quadrante, exibe o alvo no radar e registra a detecção; (3) em paralelo, as câmeras fornecem quadros de vídeo, e o módulo de visão classifica o alvo por cor, formato e tamanho; (4) a aplicação seleciona, entre as torres armadas disponíveis, a mais adequada para o alvo, priorizando as do mesmo quadrante e, entre elas, a de menor distância euclidiana (por proximidade, alvo a alvo); (5) o comando é enviado ao Arduino, que aciona a torre selecionada; (6) o acionamento demonstrativo (laser de baixa potência ou LED) só ocorre respeitando a distância mínima de segurança e as regras de zona morta, que podem bloquear um quadrante inteiro ou uma faixa de distância a partir da base; (7) toda ação executada é registrada em auditoria.

A seleção de torre e o acionamento dependem do modo de operação escolhido pelo operador, em um modelo de três níveis: no primeiro, os alvos são apenas detectados e exibidos; no segundo, acrescenta-se a seleção automática da torre, sem acionamento; no terceiro, acrescenta-se o acionamento demonstrativo automático. Quando a regra de segurança reprova um acionamento, ele é bloqueado e o motivo é registrado no console de eventos.

2.4 Diagrama de pacotes

A organização interna da aplicação desktop foi documentada por meio de um diagrama de pacotes, diagrama estrutural da UML que representa a divisão lógica do sistema em pacotes e as dependências existentes entre eles, sem detalhar atributos ou métodos das

classes neles contidas. No RadarTorres, cada pacote corresponde a um *namespace* do projeto `RadarTorres.App`, coincidente com uma pasta do código-fonte, e o diagrama foi obtido diretamente do código da *branch* principal do repositório: cada seta representa ao menos uma diretiva `using RadarTorres.App.*` (ou uma referência `clr-namespace` em XAML) presente nos arquivos do pacote de origem, de modo que o diagrama descreve a estrutura efetivamente implementada, e não apenas a pretendida.

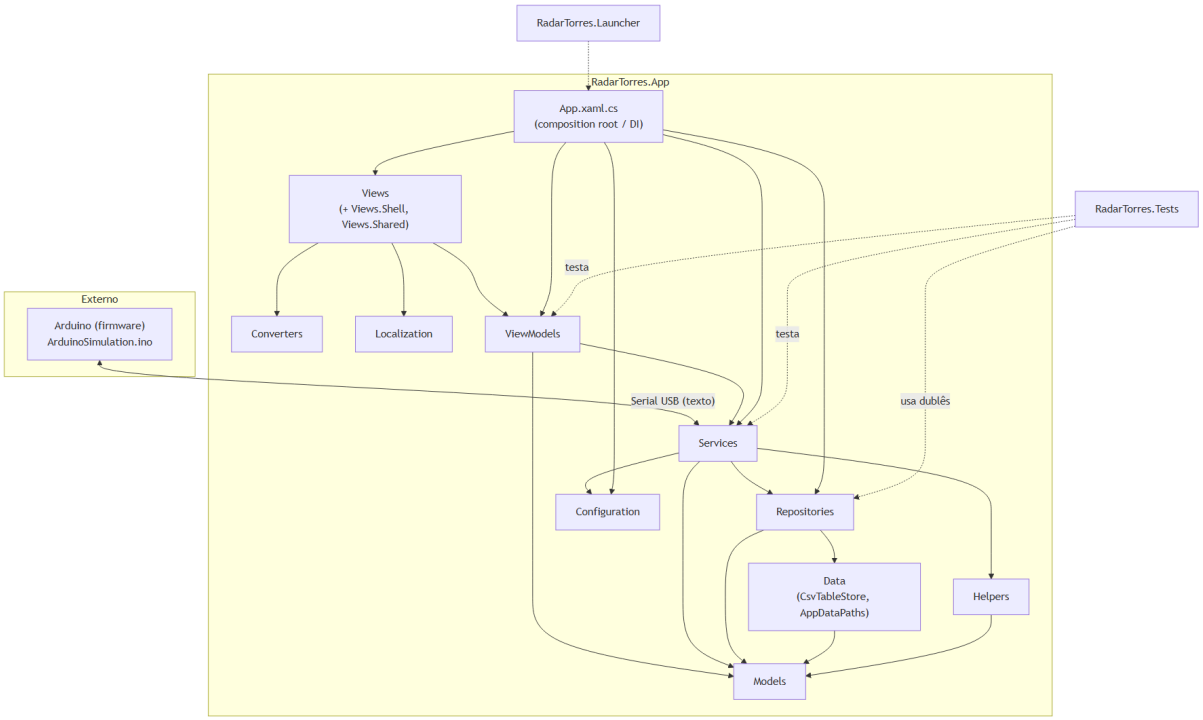


Figure 1: Diagrama de pacotes do RadarTorres. Setas em vermelho indicam dependências que formam ciclos (dos autores).

O fluxo principal de dependências segue a organização do padrão MVVM, de cima para baixo: Views → ViewModels → Services → Repositories → Data → Models. O pacote Models ocupa a posição mais estável do sistema – todos os demais dependem dele e ele não depende de nenhum outro –, o que é desejável para as entidades de domínio, e a separação entre as interfaces de Repositories e a implementação em CSV do pacote Data é o que viabiliza a futura migração para um banco relacional sem alterar ViewModels ou Services. A construção do diagrama a partir do código revelou, também, três dependências que formam ciclos entre pacotes – destacadas em vermelho na figura –: o `NavigationService` importa *namespaces* de Views para instanciar telas (Services → Views), e o `DataSeeder`, responsável pela carga inicial de dados, depende de Repositories e de Services, enquanto esses mesmos pacotes dependem de Data. Esses ciclos não impedem o funcionamento da aplicação, pois todos os pacotes pertencem ao mesmo *assembly*, mas aumentam o acoplamento e dificultam testes isolados e futuras separações em projetos distintos.

2.5 Modelo de dados

A persistência do RadarTorres foi igualmente documentada por meio de um levantamento do modelo de dados diretamente a partir do código-fonte já implementado. O protótipo

não utiliza, no estágio atual, um sistema gerenciador de banco de dados relacional: seis entidades – usuários, objetos detectados, ações realizadas, trocas de modo de operação, preferências de usuário e chamados de suporte – são persistidas localmente em arquivos CSV, um por tabela, cada uma acessada exclusivamente por uma interface de repositório dedicada, o que isola as camadas de ViewModel e Service do formato de armazenamento.

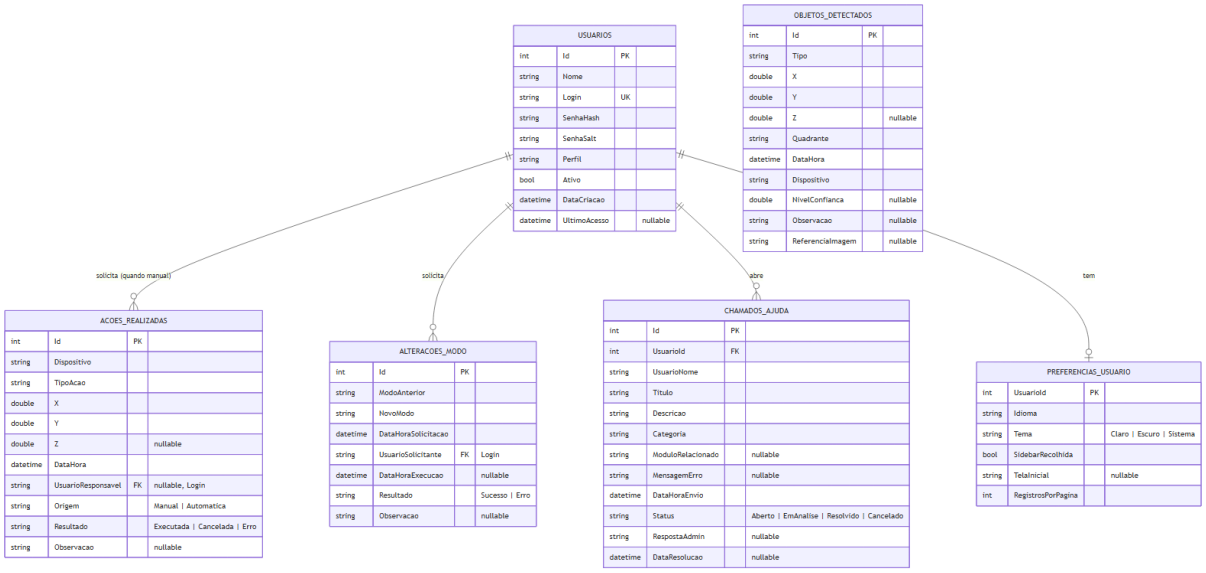


Figure 2: Modelo de dados atual (persistência em CSV) – diagrama entidade-relacionamento levantado a partir do código-fonte do projeto (dos autores).

`Usuario` se relaciona 1:1 com `PreferenciasUsuario` e 1:N com `AcaoRealizada` e `ModoAtualTorre` – nesses dois últimos casos por **login** (texto), sem chave estrangeira real – e com `ChamadoAjuda`, por identificador numérico. `ObjetoDetectado` não possui nenhum vínculo persistido com as demais entidades, sendo registrado de forma independente. Como proposta de evolução, foi elaborado um modelo relacional equivalente, no qual as referências textuais por login seriam substituídas por chaves estrangeiras íntegras por identificador numérico, com integridade garantida pelo próprio SGBD, permitindo migrar a persistência de CSV/JSON para SQL sem exigir alterações nas camadas de ViewModel ou Service, já isoladas do formato de armazenamento por meio das interfaces de repositório.

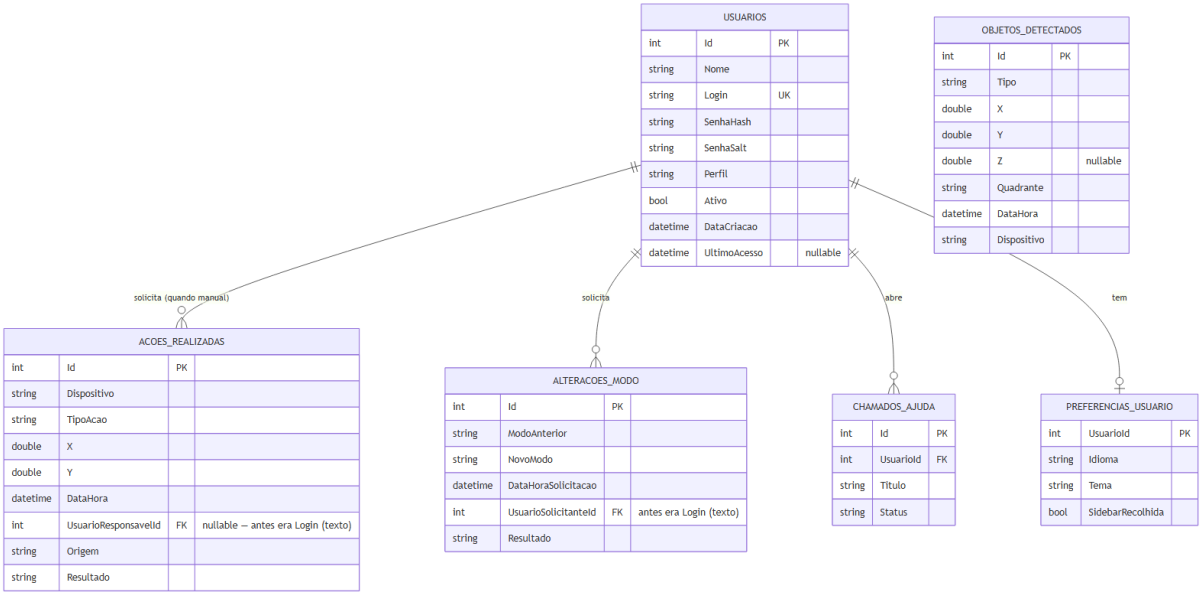


Figure 3: Modelo relacional proposto para a migração futura de persistência, de CSV/JSON para SQL (dos autores).